

# knn 근사 알고리즘을 활용한 GPT2 성능 향상 연구

김예나\*, 박준형\*, 안승현\*

Enhancing GPT-2 Performance via k-NN Approximation Algorithms

Yeana Kim\*, Junhyeong Park\*, Seunghyeon An\*

## 요약

최근 자연어처리(NLP) 분야의 발전은 대규모 언어 모델의 등장으로 가속화되었으며, 그중 GPT-2 는 자연스러운 문장 생성에서 우수한 성능을 보인다. 그러나 기존 언어 모델은 학습 데이터에 없는 외부 지식이나 새로운 문맥을 유연하게 반영하는 데 한계가 있다. 본 논문에서는 이러한 한계를 보완하기 위해 사전학습된 GPT-2 에 k-최근접 이웃(kNN) 기반의 외부 메모리 접근 방식을 결합하는 방법을 제안하며, FAISS 기반의 효율적인 검색을 통해 문맥 적응성과 일반화 성능을 향상시킨다. 데이터 디렉토리에 포함된 WikiText-2, SST, CFIMDB, Quora, Sonnet 데이터셋을 GPT tokenizer 로 전처리하고 학습하여 생성한 .pt 데이터스토어를 활용하였다. Sonnet 생성과 Quora 기반 문장 패러프레이징 실험에서, 제안한 kNN-GPT2 는 기존 Mini GPT-2 에 비해 더 자연스럽고 문맥에 부합하는 문장을 생성하였다.

## Abstract

Recent advances in natural language processing (NLP) have been driven by large-scale language models like GPT-2, which excels at generating fluent text. However, such models struggle to incorporate external knowledge or adapt to unseen contexts. We propose enhancing GPT-2 with a k-nearest neighbor (kNN) external memory, efficiently implemented via FAISS, to improve adaptability and generalization. We use data from WikiText-2, SST, CFIMDB, Quora, and Sonnet corpora, preprocessed with the GPT tokenizer and stored in .pt format for efficient retrieval. Experiments on sonnet generation and Quora paraphrasing tasks demonstrate that our method produces more natural and contextually appropriate text compared to the baseline Mini GPT-2.

## Key words

Natural Language Processing, GPT-2, k-Nearest Neighbor, External Memory, Language Model Adaptation, FAISS, DataStore, Sonnet Generation, Paraphrasing

## I. 서론

### 1. 1 연구 목적

최근 자연어처리(NLP) 분야는 대규모 언어 모델(Language Model)의 발전을 중심으로

급격히 성장하고 있다. 그중 GPT-2 는 대량의 데이터를 기반으로 자연스러운 문장 생성을 수행하며 다양한 작업에서 우수한 성능을

보이고 있다. 그러나 이러한 모델은 학습 데이터 외부의 지식이나 새로운 문맥을 유연하게 반영하지 못한다는 한계를 가진다. 이를 보완하기 위해 기존 언어 모델에 K-최근접 이웃(K-Nearest Neighbor, kNN) 기반 외부 메모리를 결합하여 언어 모델의 성능을 보완하려는 시도가 제안되었으며, 일부 연구에서 그 가능성이 탐구되고 있다.

본 논문은 GPT-2 에 kNN 기반 외부 메모리 접근 방식을 결합하여 문맥 적응성과 일반화 성능을 향상시키는 방법을 제안한다.

## 1.2 연구 범위

이 논문은 사전학습된 GPT-2 모델에 kNN 모듈을 결합하여 성능을 보완하는 방법을 탐구한다. WikiText-2, SST, CFIMDB, Quora, Sonnet 등 다양한 데이터셋에서 k 값 등 일부 설정을 변경하며 실험을 수행하고, 문맥 적응성과 일반화 성능에 미치는 영향을 분석한다.

## 1.3 연구 방법

기존 GPT-2 모델의 출력을 외부 메모리의 임베딩 공간과 비교하여 가장 유사한 k 개의 이웃을 검색하고, 이 이웃들로부터 예측 분포를 보정한다(예: 보팅, 확률 평균). 실험은 WikiText-2, SST, CFIMDB, Quora, Sonnet 데이터셋을 기반으로 수행되며, 각 데이터셋 특성에 따라 accuracy, macro F1, CHRF score 등의 지표로 성능을 평가한다.

# II. 본 론

## 2.1 GPT-2 언어 모델

GPT-2 는 OpenAI 에서 개발한 사전학습 기반 언어 생성 모델로, Transformer 아키텍처를 사용한다. Transformer 는 Attention 메커니즘을 중심으로 설계되어, 순차적인 RNN 계열 모델보다 병렬처리에 유리하고 긴 문맥 정보를 효율적으로 학습할 수 있는 구조를 가진다. GPT-2 는 Decoder-only 구조를 채택하며, 언어 생성에 최적화된 causal (unidirectional) self-attention 을

사용하여 이전 토큰들로부터 다음 토큰을 예측하는 방식으로 작동한다.

GPT-2 는 방대한 텍스트 데이터(예: 웹 크롤링 데이터)를 통해 unsupervised 방식으로 학습되며, 이후 다양한 다운스트림 작업에 활용된다. 이 모델은 문장 생성, 요약, 번역, 질의응답 등에서 우수한 성능을 보여준다. 그러나 GPT-2 는 사전학습 시 학습한 데이터 외부의 지식을 유연하게 반영하기 어렵고, 학습 데이터의 분포에 기반한 예측에 한정된다는 한계를 가진다.

## 2.2 knn 알고리즘

kNN 알고리즘(k-nearest neighbors)은 주어진 데이터 포인트와 가장 가까운 k 개의 이웃을 찾아 이들로부터 예측값을 도출하는 대표적인 instance-based 학습 방법이다. 분류 문제에서는 이웃들의 다수결 투표 방식을 통해 최종 클래스를 결정하며, 회귀 문제에서는 이웃들의 값 평균을 사용하여 예측값을 산출한다. 이 알고리즘은 데이터 분포에 대한 명시적인 가정을 하지 않고, 메모리 기반으로 작동하기 때문에 새로운 데이터에 유연하게 대응할 수 있다는 장점이 있다.

## 2.3 kNN-GPT2 구조

kNN-LM 은 기존의 언어 모델(예: GPT-2)의 출력을 외부 메모리(데이터스토어)와 결합하여 성능을 향상시키는 구조이다. 이 구조는 기존 언어 모델의 확률 분포에 더해, kNN 검색을 통해 얻은 예측 분포를 융합하는 방식으로 동작한다. 이러한 구조는 특히 훈련 데이터에 없는 문맥이나 희귀 단어에 대해 더욱 강건한 예측을 가능하게 한다.

데이터스토어는 다음과 같은 방식으로 구축된다. 1. 입력 문장들을 GPT-2 로 임베딩하여 벡터 공간에 저장 2. 각 벡터에 대응하는 출력 토큰 또는 감정 라벨 등 레이블 정보를 함께 저장 3. 검색 속도 향상을 위해 FAISS 라이브러리를 활용해 인덱스를 구성 4. 언어 모델의 예측 확률과 knn 으로부터 얻은 예측 확률을 다음 공식을 활용해 결합한다.

$$P(x) = \lambda \cdot P_{kNN}(x) + (1 - \lambda) \cdot P_{LM}(x)$$

본 연구에서 제안하는 kNN-GPT2 모델은

사전학습된 GPT-2 언어 모델에 외부 메모리 기반의 kNN 검색 기능을 결합한 하이브리드 구조로 설계되었다. 이 구조는 GPT-2의 출력 임베딩을 기준으로 외부 메모리에서 가장 유사한 k 개의 이웃을 검색하고, 이들 이웃으로부터 획득한 확률 분포를 GPT-2의 확률 분포와 결합함으로써 문맥에 존재하지 않는 희귀 단어 처리 및 새로운 지식 반영에 강건한 예측을 가능하게 한다.

외부 메모리의 임베딩 인덱싱 및 탐색은 FAISS 라이브러리를 활용하여 대규모 데이터에서도 실시간 유사도 탐색이 가능하도록 구현되었으며, 이러한 결합은 모델의 일반화 성능을 향상시키는 실용적인 접근법이다

## 2.4 실험 환경

본 연구의 실험은 Python 3.8 환경에서 수행되었으며, 주요 라이브러리로는 PyTorch, FAISS, tqdm, Einops 등을 사용하였다. 실험은 NVIDIA RTX 3060 GPU(12GB VRAM)를 탑재한 컴퓨터에서 진행되었으며, 학습 및 평가 시 동일한 하드웨어 환경을 유지하였다. 학습 과정에서는 에폭(epoch) 수는 10, 학습률은 1e-5를 사용하였다.

표 1. 실험 데이터셋 통계

| 데이터셋       | 태스크        | 학습 데이터 수 | 평가 데이터 수 |
|------------|------------|----------|----------|
| CFIMDB     | 분류(감정)     | 1706     | 1706     |
| SST        | 분류(감정)     | 8543     | 2210     |
| Quora      | 분류(패러프레이즈) | 283011   | 283003   |
| Wikitext-2 | 언어 모델링     |          |          |
| Sonnets    | 생성 (시)     | 131      | 12       |

## 2.5 sonnet 생성 실험 과정 및 결과

본 연구에서는 k-NN 증강 모델이 Sonnet 생성 성능에 미치는 영향을 분석하기 위해 Sonnet 데이터셋을 활용한 실험을 수행하였다. 실험은 sonnet\_generation.py(기본 모델) 및 sonnet\_generation\_knn.py(k-NN 증강 모델) 스크립트를 통해 진행되었으며, 두 스크립트는 GPT-2 기반 Sonnet 생성 모델을 로드하고 텍스트 생성을 수행하도록 설계되었다.

### 2.5.1 실험 설정

실험에 사용된 k-NN 증강 Sonnet 생성 모델은 사전 학습된 GPT-2 기반 SonnetGPT에 k-NN 검색기를 결합한 형태이다. 데이터스토어는 Sonnet 데이터셋의 학습 데이터로부터 추출한 768 차원의 hidden state와 해당 토큰 레이블 값으로 구성되었다. 학습은 cross-entropy loss를 최소화하도록 최적화되었으며, 평가 시에는 k-NN 검색 결과를 기반으로 출력 확률 분포를 보정하였다. 주요 하이퍼파라미터는 표 2와 같다.

표 2. 하이퍼파라미터 값

| 하이퍼파라미터     | 값    |
|-------------|------|
| k           | 4    |
| lambdaknn   | 0.4  |
| temperature | 1.2  |
| 배치 크기       | 4    |
| 학습률         | 1e-5 |
| epoch       | 10   |

데이터스토어는 GPU 메모리 효율성을 고려하여 torch tensor 형태로 저장되었으며, GPU 가용 시 GPU 메모리에 적재되었다.

### 2.5.2 실험 과정

실험은 다음 단계로 진행되었다.

1. 모델 및 데이터스토어 로드: 사전 학습된 SonnetGPT 모델을 불러오고 Sonnet 데이터 기반 데이터스토어를 메모리에 로드하였다.

2. 텍스트 생성: 학습 완료된 모델을 이용해 소넷의 빈 부분을 채우는 방식으로 텍스트를 생성하였다.
3. 평가 지표 계산: 생성된 소넷에 대해 CHRF 점수를 측정하여 품질을 평가하였다.

### 2.5.3 실험 결과

k-NN 증강 Sonnet 생성 모델은 CHRF 점수에서 기존 SonnetGPT 모델 대비 성능 향상을 보였다.

표 3. CHRF score 비교 결과

| Model     | CHRF score |
|-----------|------------|
| Mini-GPT2 | 27.06      |
| kNN-GPT2  | 40.12      |

### 2.6 paraphrase 감지 실험 과정 및 결과

본 연구에서는 k-NN 증강 모델이 패러프레이즈 탐지 성능에 미치는 영향을 분석하기 위해 Quora Question Pairs 데이터셋을 활용한 실험을 수행하였다. 실험은 `run_knn_augmented.py` 스크립트를 통해 진행되었으며, 이 스크립트는 k-NN 증강 GPT-2 모델을 로드하고 데이터스토어를 기반으로 최근접 이웃 검색을 수행하도록 설계되었다.

#### 2.6.1 실험 설정

실험에 사용된 k-NN 증강 모델은 사전 학습된 GPT-2 기반의 ParaphraseGPT 모델에 k-NN 검색기를 결합한 형태이다. 데이터스토어는 학습 데이터로부터 추출된 768 차원의 hidden state 와 해당 레이블 값으로 구성되었으며, Quora 데이터셋에서 구축되었다. 모델 학습은 cross-entropy loss 를 최소화하도록 최적화되었으며, 평가 시에는 k-NN 검색 결과를 기반으로 출력 확률 분포를 보정하였다. 주요 하이퍼파라미터는 표 3 과 같다.

표 3. 하이퍼파라미터 값

| 하이퍼파라미터 | 값  |
|---------|----|
| k       | 32 |

|             |      |
|-------------|------|
| lambdaknn   | 0.5  |
| temperature | 10.0 |
| 배치 크기       | 8    |
| 학습률         | 5e-5 |
| epoch       | 3    |

데이터스토어는 GPU 메모리 효율성을 고려하여 청크 단위로 로드되었으며, 최대 5 개의 청크까지 메모리에 탑재되도록 설정하였다.

#### 2.6.2 실험 과정

실험은 다음 단계로 진행되었다.

1. 모델 및 데이터스토어 로드: 사전 학습된 ParaphraseGPT 모델을 불러오고, Quora 데이터 기반 데이터스토어를 메모리에 로드하였다. 데이터스토어는 torch tensor 로 구성되며, GPU 가용 시 GPU 메모리에 적재되었다.
2. 테스트 데이터셋 로드: Quora 테스트 데이터셋(quora-test-student.csv)을 로드하여 tokenization 및 attention mask 생성 후 DataLoader 로 배치화하였다.
3. k-NN 증강 탐지 수행: 각 테스트 배치에 대해 k-NN 검색 결과를 활용하여 출력 logits 을 보정한 후, softmax 를 적용하여 최종 예측 라벨을 도출하였다.

#### 2.6.3 실험 결과

k-NN 증강 모델의 성능은 기존 ParaphraseGPT 단독 모델과 비교하여 다음과 같은 성능을 보였다.

| Model     | 정확도 (dev acc) | macro F1 score |
|-----------|---------------|----------------|
| Mini-GPT2 | 0.896         | -              |
| kNN-GPT2  | 0.8981        | 0.8918         |

### III. 결 론

#### 3.1 sonnet 생성 결과 분석 및 개선방향

CHRF (Character n-gram F-score)는 자연어 생성에서 생성 문장의 문자를 n-gram 단위로 분석하여 정확도와 재현율을 평가하는 지표이다. 이는 단어 단위의 BLEU 점수보다 문자 간 유연성을 더 잘 반영할 수 있다. 최소값은 0에서 시작하여 최대값은 100이다.

결과적으로 CHRF 점수는 기존 27.06에서 40.12로 크게 향상되었다. 이는 KNN 방식을 통해 외부 메모리에서 유사 문장을 참고하여 sonnet을 생성했기 때문으로 보인다. 특히, 외부 메모리가 기존 sonnet과 유사한 참조 데이터였기에 점수 향상에 유리하게 작용했음을 시사한다.

다만, 상대적으로 점수가 향상되었음에도 최대값인 100 점수를 고려하면 절대적인 점수 수준은 여전히 만족스럽지 않다. 또한 KNN 방식을 활용한 결과, 정확도는 높아졌지만 창의성 측면에서는 한계가 있었다. 더 중요한 과제는 Openweb, Amazon reviews, yelp reviews와 같은 다른 데이터셋에서도 높은 품질의 sonnet을 생성하고 좋은 점수를 얻을 수 있는지 여부이다. 향후에는 epoch, 학습률, 메모리 크기, neighbor 수 등 변수를 체계적으로 조정하고 데이터 양을 충분히 확보하는 한편, BERTScore와 같은 문맥 기반 평가 지표를 도입하여 품질을 더욱 향상시킬 수 있을 것이다.

#### 3.2 paraphrase 감지 결과 분석 및 개선방향

Paraphrase 감지 실험에서는 k-NN 증강 ParaphraseGPT 모델을 Quora Question Pairs 데이터셋 기반으로 평가하였다. 실험 결과, kNN-GPT2 모델은 dev accuracy 0.8981, F1 score 0.8918을 기록하였으며, 이는 기존 Mini-GPT2 단독 모델의 dev accuracy 0.896에 비해 미미하지만 성능 향상이 있었다. 실험에 사용한

데이터셋은 원래 구축하려던 대용량 paraphrase 쌍이 포함된 새로운 데이터셋 대신, 시간 제약으로 인해 Wikitext가 포함된 기존 데이터를 사용하였으며, 이는 성능 향상 폭이 제한적인 원인 중 하나로 판단된다.

또한, 데이터스토어 로드시 청크 개수를 3개로 제한한 것도 성능에 영향을 미친 요소였다. 실험 환경이 허락되었다면 8개 이상으로 청크를 늘려 더 많은 데이터를 활용하는 것이 가능했을 것이며, 이는 검색 정확도와 모델 성능 향상에 기여할 수 있었을 것으로 예상된다.

실험 과정에서 k, lambda, temperature 값과 같은 주요 하이퍼파라미터를 고정하여 다양한 조합에 따른 성능 변화를 충분히 탐색하지 못한 점도 아쉬움으로 남는다. 향후 개선을 위해서는 paraphrase 데이터스토어를 Quora 데이터 외에도 다양한 도메인의 paraphrase 쌍으로 확장해 모델의 일반화 능력을 높이고, 하이퍼파라미터를 grid search나 AutoML 도구를 통해 체계적으로 최적화하는 것이 필요하다.

또한, 현재 torch tensor 기반의 검색 방식을 ScaNN과 같은 고속 근사 최근접 탐색 라이브러리로 대체하면 검색의 정확성과 효율성을 동시에 개선할 수 있을 것이다. 마지막으로, dev accuracy와 Macro F1 score 외에도 ROC-AUC, PR-AUC 등 다양한 지표를 포함해 성능을 다각도로 분석하고, confusion matrix 및 성능 변화 그래프를 활용해 k-NN 증강의 영향을 정밀하게 파악하는 것이 바람직하다.

### 3.3 결 론

본 논문에서는 사전학습된 GPT-2 언어 모델에 kNN 근사 알고리즘을 결합하여, 외부 메모리 기반의 유사도 검색을 통해 희귀 단어나 훈련 데이터에 존재하지 않는 문맥에 대한 예측

성능을 향상시킬 수 있음을 실험적으로 보였다. 제안한 kNN-GPT2 구조는 기존 GPT-2 에 비해 더 강건하고 유연한 문장 생성을 가능하게 하며, 특히 훈련 데이터의 한계를 보완하는 효과를 확인하였다.

향후 연구에서는 kNN 기반의 동적 컨텍스트 학습, 메타러닝과의 결합, 또는 하이브리드 메커니즘을 통한 메모리 효율 최적화 등 다양한 확장 가능성을 모색할 수 있다. 또한, 실제 애플리케이션(예: 대화형 AI, 문서 요약)에서의 활용성과 실시간 응답 속도를 개선하기 위한 추가 연구도 필요하다.